

ACE ENGINEERING COLLEGE

An AUTONOMOUS Institution

NBA Accredited B. Tech Courses, Accorded NAAC 'A' Grade

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, Telangana) Ankushapur
(V), Ghatkesar (M), Hyderabad - 501 301.

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (IOT)

PYTHON PROGRAMMING MICRO-PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology (B.Tech)

in

COMPUTER SCIENCE ENGINEERING (IOT)

on the micro-project titled

SMART STUDY COMPANION: MASTER YOUR LEARNING

Submitted by

Roll Number	Student Name
25AG1A6915	Sai Srinidhi
25AG1A6925	Ruthwik
25AG1A6912	Hasmitha
25AG1A6951	Krishna Kishore

Under the Guidance of

Mrs.Y Durga Bhargavi
(Assistant professor)

Academic Year: 2025 – 2026

ACE ENGINEERING COLLEGE

An AUTONOMOUS Institution

NBA Accredited B. Tech Courses, Accorded NAAC 'A' Grade
(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, Telangana)
Ankushapur (V), Ghatkesar (M), Hyderabad - 501 301.

CERTIFICATE

This is to certify that the Micro Project work entitled “**SMART STUDY COMPANION: MASTER YOUR LEARNING**” is being submitted by **Sai Srinidhi (25AG1A6915), Ruthwik (25AG1A6925), Hasmitha (25AG1A6912), and Krishna Kishore (25AG1A6951)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Internet of Things) to Jawaharlal Nehru Technological University, Hyderabad, during the academic year 2025–26.

This project work is a record of bonafide work carried out by the above students under the guidance and supervision of **Mrs. Y Durga Bhargavi**, Assistant Professor, Department of Computer Science and Engineering (IoT), **ACE Engineering College**.

The results embodied in this report have not been submitted by the students to any other University or Institution for the award of any degree or diploma.

Internal Guide

Mrs.Y Durga Bhargavi
Assistant Professor

Head of the department

Dr.M.V.Vijaya Saradhi
Professor and Dean HoD CSE(IoT)

ACKNOWLEDGEMENT

We express our sincere gratitude to all those who have directly or indirectly contributed to the successful completion of our Micro Project titled “**SMART STUDY COMPANION: MASTER YOUR LEARNING**”. First and foremost, we would like to thank our parents for their constant encouragement, support, and motivation throughout our academic journey. Their guidance and belief in our abilities have been a source of strength and inspiration.

We extend our heartfelt thanks to our General Secretary, **Prof. Y. V. Gopala Krishna Murthy**, for establishing and nurturing such a prestigious institution that provides students with excellent opportunities for learning and innovation.

We are grateful to our respected Principal, **Dr. N. Sudhakar Reddy**, for providing the facilities and environment necessary for carrying out this project successfully.

We express our sincere gratitude to **Dr. M. V. Vijaya Saradhi**, Professor, Dean and Head of the Department of Computer Science and Engineering (Internet of Things), for his valuable encouragement, guidance, and support throughout the development of this project.

We are especially thankful to our project guide, **Mrs. Y Durga Bhargavi**, Assistant Professor, Department of CSE (IoT), whose continuous guidance, constructive suggestions, and technical expertise helped us complete this project successfully. Her encouragement and support played a significant role in every stage of the project.

We also thank all the faculty members and non-teaching staff of the Department of Computer Science and Engineering (IoT) for their valuable assistance and cooperation whenever required.

Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project.

25AG1A6915 – Sai Srinidhi

25AG1A6925 – Ruthwik

25AG1A6912 – Hasmitha

25AG1A6951 – Krishna Kishore

INDEX

Table of Contents

TITLE	1
CERTIFICATE.....	2
ACKNOWLEDGEMENT	3
ABSTRACT	5
INTRODUCTION	6
OBJECTIVES	7
SOFTWARE REQUIREMENTS	8
MODULES USED	9
ALGORITHM	10
SOURCE CODE	12
OUTPUT SCREENS	24
TEST CASES AND RESULTS	28
ADVANTAGES	29
CONCLUSION	30
FUTURE ENHANCEMENTS	31
REFERENCES	32

ABSTRACT

In today's fast-paced academic environment, students often struggle with inefficient study habits, poor time management, and the inability to retain what they have studied. The absence of any structured mechanism to track study sessions and reinforce learned content results in wasted hours, burnout, and poor academic performance. To address these challenges, this project presents the Smart Study Companion – a Python-based desktop application designed to help students track, measure, and improve their study sessions in a data-driven manner.

The Smart Study Companion integrates three core modules: the Focus Tracker, which records the start and end time of each study session, calculates total study duration, and monitors break intervals; the Quiz Mode, which automatically generates a set of topic-relevant questions at the end of a session to reinforce retention; and the Performance Analytics module, which visualizes study patterns, quiz scores, and optimal study hours through graphs and trend charts.

The application is developed entirely in Python 3.x using standard libraries such as datetime for session time management, json for persistent data storage, random for quiz generation, and matplotlib for performance visualizations. The system is designed to run as a console-based tool, making it lightweight, portable, and easy to deploy across all platforms. The project was built as part of the Python Programming coursework and demonstrates the practical application of Python concepts in solving realworld academic challenges. The outcomes of this project include improved study awareness, better knowledge retention, and a measurable increase in study efficiency for its users.

INTRODUCTION

In the current academic landscape, students at every level are expected to manage an increasing volume of coursework, assignments, and examinations. Despite having access to a wide range of learning resources, many students fail to develop structured study habits that allow them to make the most of their available time. A common observation is that students spend hours sitting in front of their books or screens, but a large portion of that time is unproductive due to distractions, poor planning, and the absence of any feedback mechanism to evaluate the effectiveness of their study sessions.

One of the biggest problems students face is that they have no visibility into how much time they are spending on studying versus browsing social media or taking extended breaks. Without this awareness, it is almost impossible to identify and address inefficiencies in one's study routine. Furthermore, even when students do spend quality time studying, the lack of immediate reinforcement often leads to rapid forgetting of the material, a phenomenon well-documented in educational psychology through the concept of the forgetting curve.

The Smart Study Companion is proposed as a practical solution to these challenges. It is a Python-based application that empowers students to take control of their learning by providing three key capabilities: time tracking, knowledge reinforcement through quizzes, and performance visualization. The application is designed to be simple enough for any student to use without technical knowledge, yet powerful enough to provide meaningful insights into their study habits over time.

The project was conceived with the understanding that technology, when applied thoughtfully, can serve as a powerful academic aid. Rather than being a distraction, a well-designed study companion can help students become more self-aware about their learning patterns and take corrective action to improve. By integrating session tracking, automated quizzes, and graphical analytics into a single tool, the Smart Study Companion aims to bridge the gap between effort and performance in student learning.

This project is implemented using Python 3.x, one of the most widely taught and used programming languages in academic and professional settings. It makes use of several built-in and standard Python libraries, which are described in detail in the subsequent chapters. The development of this application involved understanding core Python concepts such as file handling, data structures, modules, object-oriented programming, and data visualization, making it a comprehensive learning exercise as well as a functional product.

OBJECTIVES

The primary objective of the Smart Study Companion project is to design and develop a Python-based application that helps students improve their study habits through time tracking, knowledge testing, and performance analysis. The specific objectives of the project are as follows:

- To develop a Focus Tracker module that records the start time, end time, total duration, and break time of each study session.
- To implement a Quiz Mode feature that automatically generates topic-specific questions after the completion of a study session to reinforce knowledge retention.
- To build a Performance Analytics module that visualizes study trends, quiz scores, and optimal study hours using graphical representations.
- To persist session data locally using file-based storage so that students can view their historical study records.
- To design the application in a modular and maintainable manner using Python's standard programming practices.
- To provide a simple, intuitive, and console-based user interface that any student can operate without prior technical knowledge.
- To demonstrate the practical application of Python programming concepts including datetime manipulation, file I/O, data structures, random number generation, and data visualization.
- To produce a tool that promotes a data-driven approach to studying and encourages students to develop consistent and productive study habits.

SOFTWARE REQUIREMENTS

The following software tools and technologies were used in the development and execution of the Smart Study Companion project

Programming Language

- Python 3.x (Version 3.8 or higher recommended)

Development Environment

- Visual Studio Code (VS Code) with Python Extension
- PyCharm Community Edition
- IDLE (Python's built-in Integrated Development Environment)

Operating System

- Windows 10 / 11
- Linux (Ubuntu 20.04 or later)
- macOS (Monterey or later)

Python Libraries Required

- datetime – for managing and calculating study session times
- json – for storing and retrieving session data in JSON format
- os – for file path management and checking file existence
- random – for generating quiz questions and randomizing options
- matplotlib (version 3.5+) – for generating study analytics graphs • collections – for organizing and processing session data

Installation Commands

```
pip install matplotlib
```

All other modules used in the project are part of Python's standard library and do not require separate installation.

MODULES USED

The Smart Study Companion application is built using a combination of Python's built-in standard library modules and one third-party visualization library. Each module plays a specific role in supporting the functionality of the application. The following is a detailed description of each module used in the project.

1. datetime Module

The datetime module is one of the most important components of the Focus Tracker feature. It is used to capture the precise start time and end time of each study session. The timedelta class within this

module is used to calculate the total duration of the session as well as the duration of individual breaks. All time values are stored in ISO format strings for compatibility with the JSON storage module.

2. json Module

The json module handles all data persistence in the application. When a study session is completed, the session data including the topic, start time, end time, duration, break count, and quiz score is serialized into JSON format and appended to a local file named study_sessions.json. This file acts as the application's database and is read each time the user wishes to view their performance analytics.

3. os Module

The os module is used to check whether the session data file already exists on the disk before attempting to read from it. This prevents file-not-found errors and ensures that the application handles first-time execution gracefully by creating a new data file if none is present.

4. random Module

The random module supports the Quiz Mode feature of the application. After the user completes a study session on a given topic, a set of predefined questions for that topic are stored in a dictionary. The random module is used to randomly select 3 to 5 questions from the available pool and to shuffle the answer choices so that the correct answer does not always appear in the same position.

5. matplotlib.pyplot Module

The matplotlib.pyplot module is used to generate graphical representations in the Performance Analytics section. It produces a bar chart showing the number of study hours logged per day, a line graph displaying quiz score trends over multiple sessions, and a histogram identifying the hours of the day during which the student studies most frequently. These visualizations help students understand their learning patterns at a glance.

6. collections.Counter

The Counter class from the collection's module is used in the analytics module to efficiently count and aggregate data, such as calculating how many sessions were held on each day of the week or how many times a particular study hour appears in the session records.

ALGORITHM

The Smart Study Companion application operates through three main functional workflows: the Focus Tracker, the Quiz Mode, and the Performance Analytics. The following algorithms describe the stepby- step logic of each module.

Algorithm 1: Focus Tracker

- Step 1:** Start the program and display the main menu to the user.
- Step 2:** Accept the study topic name as input from the user.
- Step 3:** Record the current system time as the session start time using `datetime.now()`.
- Step 4:** Begin the study session. Display a message prompting the user to press Enter to log a break or end the session.
- Step 5:** If the user logs a break, record the break start time. When the user resumes, calculate and add the break duration to the total break time.
- Step 6:** When the user ends the session, record the current time as the session end time.
- Step 7:** Calculate total session duration = end time - start time.
- Step 8:** Calculate net study time = total duration - total break time.
- Step 9:** Store the session details (topic, start, end, net duration, break count) in the JSON data file.
- Step 10:** Display a session summary to the user and proceed to Quiz Mode.
- Step 11:** Stop.

Algorithm 2: Quiz Mode

- Step 1:** Start the Quiz Mode immediately after the study session ends.
- Step 2:** Retrieve the topic studied during the session.
- Step 3:** Load the question bank for that topic from the predefined dictionary.
- Step 4:** Randomly select 3 to 5 questions from the question bank.
- Step 5:** For each question, shuffle the answer options and display them to the user.
- Step 6:** Accept the user's answer as input.
- Step 7:** Compare the answer with the correct option and record whether it is correct or incorrect.
- Step 8:** Repeat steps 5 to 7 for all selected questions.
- Step 9:** Calculate the quiz score as $(\text{correct answers} / \text{total questions}) \times 100$. **Step**
- 10:** Display the quiz score and save it to the session record in the JSON file. **Step**
- 11:** Stop.

Algorithm 3: Performance Analytics

Step 1: Start the Analytics module from the main menu.

Step 2: Load all session records from the JSON data file.

Step 3: Extract and aggregate study durations grouped by date.

Step 4: Plot a bar chart showing study hours per day using matplotlib.

Step 5: Extract quiz scores from all sessions in chronological order.

Step 6: Plot a line graph showing quiz score trends over time.

Step 7: Extract the start hour from each session and count occurrences using Counter.

Step 8: Plot a histogram showing the distribution of study start times to identify the most productive hours.

Step 9: Display all three charts and present a textual summary of total study time and average quiz score.

Step 10: Stop.

SOURCE CODE

The following is the complete source code for the Smart Study Companion application developed in Python 3.x. The code is organized into three main sections corresponding to the Focus Tracker, Quiz Mode, and Performance Analytics modules.

```
=== SECTION: Configuration ===
```

Always store the data file next to the script, regardless of the working directory from which the application is launched.

```
_SCRIPT_DIR = os.getcwd
```

```
DATA_FILE = os.path.join(_SCRIPT_DIR, 'study_sessions.json')
```

```
=== SECTION: Data Persistence ===
```

```
def load_sessions() -> list:
```

```
    """
```

Load all study sessions from the JSON data file.

Returns an empty list if the file does not exist or is corrupt, preventing crashes on first launch or after file corruption.

```
    """
```

```
    if not os.path.exists(DATA_FILE):
```

```
        return []
```

```
    try:
```

```
        with open(DATA_FILE, 'r', encoding='utf-8') as f:
```

```
            data = json.load(f)
```

```
            # Guarantee we always return a list even if the top-level
```

```
            # JSON type was accidentally changed (e.g. {} instead of []).
```

```
            return data if isinstance(data, list) else []
```

```
    except (json.JSONDecodeError, OSError) as exc:
```

```
        print(f'[Warning] Could not read session data ({exc}). '
```

```

        'Starting with an empty session history.')
    return []

```

def save_session(session: dict) -> bool:

```

"""

```

Append a completed session record to the JSON data file.

Uses an atomic write pattern (write to a temp file, then rename)

so a crash or power loss during the write never corrupts existing data. Returns True on success, False on failure.

```

"""

```

```

sessions = load_sessions()
sessions.append(session)
tmp_file = DATA_FILE + '.tmp'
try:
    with open(tmp_file, 'w', encoding='utf-8') as f:
        json.dump(sessions, f, indent=4, ensure_ascii=False)
        os.replace(tmp_file, DATA_FILE) # atomic on POSIX; near-atomic on Windows
        return True
except OSError as exc:
    print(f'[Error] Could not save session data: {exc}')
    # Clean up the temp file if it was created.
    if os.path.exists(tmp_file):
        try:
            os.remove(tmp_file)
        except OSError:
            pass
    return False

```

=== SECTION: Module 1 — Focus Tracker ===

def start_focus_session() -> dict:

```

"""

```

Interactively run a focus-tracked study session.

Prompts the user for a topic, records start time, supports multiple break intervals, records end time, then returns a session dict ready

```
to be passed to run_quiz() and save_session().
```

```
"""
```

```
# — Topic input with validation
```

```
while True:
```

```
    topic = input('Enter the topic you are studying: ').strip()
```

```
    if topic:
```

```
        break
```

```
    print(' Topic cannot be empty. Please enter a topic name.')
```

```
print('\nSession started.')
```

```
print(' Press Enter to log a break, or type "end" and press Enter to finish.\n')
```

```
start_time = datetime.datetime.now()
```

```
total_break = datetime.timedelta(0)
```

```
break_count = 0
```

```
while True:
```

```
    user_input = input().strip().lower()
```

```
    if user_input == 'end':
```

```
        break
```

```
    elif user_input == ":
```

```
        # Blank Enter → start a break
```

```
        break_start = datetime.datetime.now()
```

```
        input(' [Break started. Press Enter to resume...])
```

```
        break_end = datetime.datetime.now()
```

```
        total_break += (break_end - break_start)
```

```
        break_count += 1
```

```
        print(f' Break {break_count} recorded. Session resumed.\n')
```

```
    else:
```

```
        # Catch any other text the user might accidentally type
```

```
        print(' Unrecognised input. Press Enter for a break or type "end" to finish.')
```

```
end_time = datetime.datetime.now()
```

```
total_duration = end_time - start_time
```

```
net_study_time = total_duration - total_break
```

```
# Guard against a negative net study time (e.g. a break longer than
# the session itself, which should not happen but is defensive).
if net_study_time.total_seconds() < 0:
    net_study_time = datetime.timedelta(0)
```

```
# — Session summary
```

```
print('\nSession Summary:')
print(f' Topic      : {topic}')
print(f' Start Time   : {start_time.strftime("%H:%M:%S")}')
print(f' End Time     : {end_time.strftime("%H:%M:%S")}')
print(f' Total Duration: {str(total_duration).split(".")[0]}')
print(f' Breaks Taken  : {break_count}')
print(f' Net Study Time: {str(net_study_time).split(".")[0]}')

session = {
    'topic'      : topic,
    'date'       : start_time.strftime('%Y-%m-%d'),
    'start_hour' : start_time.hour,
    'duration_minutes' : round(net_study_time.total_seconds() / 60, 2),
    'break_count' : break_count,
    'quiz_score' : None,
}
return session
```

=== SECTION: Module 2 — Quiz Mode (Question Bank) ===

Each entry: {'q': , 'options': [, ...], 'answer': }

'answer' MUST be one of the strings in 'options' (exact match, case-sensitive).

QUESTION_BANK: dict = {

'python': [

{

'q' : 'Which keyword is used to define a function in Python?',

'options': ['func', 'def', 'define', 'function'],

```
'answer' : 'def',

},

{

'q' : 'What is the output of type(3.14) in Python?',

'options': ['int', 'float', 'double', 'str'],

'answer' : 'float',

},

{

'q' : 'Which module is used for mathematical operations in Python?',      'options': ['calc', 'math', 'number',
'options'],

'answer' : 'math',

},

{

'q' : 'What symbol is used for single-line comments in Python?',

'options': ['//', '/*', '#', '--'],

'answer' : '#',

},

{

'q' : 'Which data type is immutable in Python?',

'options': ['list', 'dict', 'set', 'tuple'],

'answer' : 'tuple',

},
```

],

'data structures': [

{

'q' : 'Which data structure follows LIFO order?',

'options': ['Queue', 'Stack', 'Deque', 'Array'],

'answer' : 'Stack',

},

{

q' : 'What is the time complexity of binary search?',

'options': ['O(n)', 'O(n^2)', 'O(log n)', 'O(1)'],

'answer' : 'O(log n)',

},

{

'q' : 'Which structure uses key-value pairs?',

'options': ['List', 'Tuple', 'Dictionary', 'Set'],

'answer' : 'Dictionary', },], }

MIN_QUESTIONS = 3

MAX_QUESTIONS = 5

=== SECTION: Module 2 — Quiz Mode (run_quiz) ===

def run_quiz(session: dict) -> dict:

```
"""
```

Present a randomised quiz for the studied topic and record the score.

Selects between MIN_QUESTIONS (3) and MAX_QUESTIONS (5) questions from the question bank (Algorithm 2, Step 4). Answer options are shuffled each time. Invalid or out-of-range input is gracefully skipped.

Returns the session dict with 'quiz_score' populated (or None if the topic is not in the question bank).

```
"""
    topic_key = session['topic'].strip().lower()

    if topic_key not in QUESTION_BANK:

print(f'\nNo questions available for the topic {session["topic"]}'. "Skipping quiz.")

return session

pool = QUESTION_BANK[topic_key]

# Enforce both the minimum (3) and maximum (5) per the algorithm spec.
if len(pool) < MIN_QUESTIONS:
    print(f'\n[Warning] Only {len(pool)} question(s) available for '
          f'{session["topic"]}' (minimum is {MIN_QUESTIONS}). '
          'Skipping quiz.')
    return session

num_questions = min(MAX_QUESTIONS, len(pool)) # 3 ≤ num_questions ≤ 5
questions = random.sample(pool, num_questions)

print(f'\n--- QUIZ TIME: {session["topic"].upper()} ---')
score = 0

for i, q in enumerate(questions, 1):
    # Shallow-copy the options list before shuffling so the original
    # bank entry is never mutated between sessions.
    opts = q['options'][:]
    random.shuffle(opts)
```

```

num_opts = len(opts)

print(f'\nQ{i}: {q["q"]}')
for j, opt in enumerate(opts, 1):
    print(f' {j}. {opt}')

# Prompt shows the actual valid range, not a hardcoded "(1-4)".
raw = input(f'Your answer (1-{num_opts}): ').strip()

try:
    ans_idx = int(raw) - 1
    # Explicitly reject out-of-range values including negatives,
    # which Python would otherwise silently accept via negative indexing.
    if not (0 <= ans_idx < num_opts):
        raise IndexError
    if opts[ans_idx] == q['answer']:
        print(' Correct!')
        score += 1
    else:
        print(f' Wrong. Correct answer: {q["answer"]}')
except (ValueError, IndexError):
    print(f' Invalid input. Expected a number between 1 and {num_opts}. '
          'Question skipped.')

quiz_score = round((score / len(questions)) * 100, 2)
print(f'\nQuiz Score: {score}/{len(questions)} = {quiz_score}%')

session['quiz_score'] = quiz_score
return session

```

=== SECTION: Module 3 — Performance Analytics ===

Required keys in every session record. Used to skip/warn on malformed entries.

```

_REQUIRED_SESSION_KEYS = {'topic', 'date', 'start_hour', 'duration_minutes', 'break_count',
'quiz_score'}

```

```

def _validate_sessions(sessions: list) -> list:

```

```

"""

```

Return only sessions that contain all required keys.

Prints a warning for any record that is skipped.

```
"""
```

```
valid = [ ]
```

```
for idx, s in enumerate(sessions):
```

```
    missing = _REQUIRED_SESSION_KEYS - s.keys( )
```

```
    if missing:
```

```
        print(f'[Warning] Session record #{idx + 1} is missing fields '
```

```
        f'{missing} and will be excluded from analytics.')
```

```
    else:
```

```
        valid.append(s)
```

```
return valid
```

```
def show_analytics() -> None:
```

```
"""
```

Load session data and display:

1. A textual summary (total study time, average quiz score).
2. Chart 1 – Bar chart of study hours per day.
3. Chart 2 – Line graph of quiz score trend (chronological).
4. Chart 3 – Bar chart of session frequency by hour of day.

Output order matches documentation Screen 5: summary first, then charts.

```
"""
```

```

raw_sessions = load_sessions()

if not raw_sessions:

    print('No session data found. Start a study session first.')
    return

sessions = _validate_sessions(raw_sessions)
if not sessions:
    print('No valid session data to display after validation.')
    return

# — Textual summary (per documentation Screen 5: summary BEFORE charts)
total_hours = sum(s['duration_minutes'] for s in sessions) / 60
quiz_results = [s['quiz_score'] for s in sessions
                 if s['quiz_score'] is not None]
avg_score = (sum(quiz_results) / len(quiz_results)) if quiz_results else 0.0

print(f'\nTotal Study Time : {total_hours:.2f} hours')
print(f'Average Quiz Score: {avg_score:.2f}%')
print()

# — Chart 1: Study Hours per Day —————
date_totals: Counter = Counter()
for s in sessions:
    date_totals[s['date']] += s['duration_minutes'] / 60

sorted_dates = sorted(date_totals.keys())
daily_hours = [date_totals[d] for d in sorted_dates]

plt.figure(figsize=(10, 4))
plt.bar(sorted_dates, daily_hours, color='steelblue', edgecolor='black')
plt.title('Study Hours Per Day')
plt.xlabel('Date')
plt.ylabel('Hours Studied')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# — Chart 2: Quiz Score Trend (chronological order per Algorithm 3) —
scored_sessions = sorted(
    [(s['date'], s['quiz_score']) for s in sessions
     if s['quiz_score'] is not None],

```

```

key=lambda x: x[0] # sort by date string (ISO format sorts lexicographically)
)

if scored_sessions:
    score_labels = [entry[0] for entry in scored_sessions] # plain list, no f-string needed
    score_values = [entry[1] for entry in scored_sessions]

    plt.figure(figsize=(10, 4))
    plt.plot(score_labels, score_values,
             marker='o', color='darkorange', linewidth=2)
    plt.title('Quiz Score Trends Over Time')
    plt.xlabel('Session Date')
    plt.ylabel('Score (%)')
    plt.ylim(0, 105)
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print('[Info] No quiz scores recorded yet; score-trend chart skipped.')

# — Chart 3: Best Study Hours (session frequency by hour of day) —
hour_counts = Counter(s['start_hour'] for s in sessions)
hours_axis = sorted(hour_counts.keys())
session_freq = [hour_counts[h] for h in hours_axis]

plt.figure(figsize=(8, 4))
plt.bar(hours_axis, session_freq, color='mediumseagreen', edgecolor='black')
plt.title('Best Study Hours (Session Frequency)')
plt.xlabel('Hour of the Day (0–23)')
plt.ylabel('Number of Sessions')
plt.xticks(range(0, 24)) # show all hours on x-axis for clarity
plt.tight_layout()
plt.show()

=== SECTION: Main Menu ===

def main() -> None:

    """

```

Entry point. Displays the main menu and dispatches to the three core modules. Handles KeyboardInterrupt for a clean Ctrl-C exit.

```
"""
```

```
print('=' * 50) print(' SMART STUDY COMPANION: MASTER YOUR LEARNING') print('=' * 50)
```

```
try:
```

```
    while True:
```

```
        print('\n1. Start Study Session')
```

```
        print('2. View Performance Analytics')
```

```
        print('3. Exit')
```

```
        choice = input('Select an option (1-3): ').strip()
```

```
        if choice == '1':
```

```
            session = start_focus_session()
```

```
            session = run_quiz(session)
```

```
            if save_session(session):
```

```
                print('Session saved successfully.')
```

```
            else:
```

```
                print('[Warning] Session could not be saved to disk.')
```

```
        elif choice == '2':
```

```
            show_analytics()
```

```
        elif choice == '3':
```

```
            print('Thank you for using Smart Study Companion. Keep learning!')
```

```
            break
```

```
        else:
```

```
            print('Invalid option. Please enter 1, 2, or 3.')
```

```
except KeyboardInterrupt:
```

```
    # Clean exit on Ctrl-C without printing a raw traceback.
```

```
    print('\n\nSession interrupted. Goodbye!')
```

```
    sys.exit(0)
```

```
if name == 'main':
```

```
    main()
```

OUTPUT SCREENS

The following section describes the expected output screens of the Smart Study Companion application at each stage of execution. Since this is a console-based Python application, all interactions take place through the command-line terminal.

Screen 1: Main Menu

When the program is launched, the main menu is displayed as follows:

```
=====
SMART STUDY COMPANION: MASTER YOUR LEARNING
=====

1. Start Study Session
2. View Performance Analytics
3. Exit
Select an option (1-3): _
```

The user enters 1, 2, or 3 to navigate to the respective module.

Screen 2: Focus Tracker – Session in Progress

When the user selects option 1 and enters a topic, the session begins:

```
Enter the topic you are studying: Python
Session started. Press Enter to take a break or type "end" to finish.

[Break started. Press Enter to resume...]
Break 1 recorded. Session resumed.
End
```

```
if __name__ == '__main__':
    main()
    plt.title('Best Study Hours (Session Frequency)')
    plt.xlabel('Hour of the Day (0-23)')
    plt.ylabel('Number of Sessions')
    plt.xticks(range(0, 24)) # show all hours on x-axis for clarity
    plt.tight_layout()
    plt.show()

=====
SMART STUDY COMPANION: MASTER YOUR LEARNING
=====

1. Start Study Session
2. View Performance Analytics
3. Exit
Select an option (1-3): 1
Enter the topic you are studying: python

Session started.
    Press Enter to log a break, or type "end" and press Enter to finish.

[Break started. Press Enter to resume...]
Break 1 recorded. Session resumed.

end

Session Summary:
    Topic      : python
```

Screen 3: Session Summary

After the user types 'end', a complete session summary is displayed:

```
Session Summary:
  Topic          : Python
  Start Time     : 10:15:22
  End Time       : 12:30:05
  Total Duration: 2:14:43
  Breaks Taken  : 1
  Net Study Time: 2:07:18
```

Screen 4: Quiz Mode

Immediately after the session summary, the quiz is automatically presented:

```
--- QUIZ TIME: PYTHON ---
```

Q1: Which keyword is used to define a function in Python?

1. function
2. define
3. def
4. func

Your answer (1-4): 3

Correct!

Q2: Which data type is immutable in Python?

1. dict
2. tuple
3. list
4. set

Your answer (1-4): 2

Correct!

Quiz Score: 4/5 = 80.00% Session
saved successfully.

Session Summary:

```
Topic          : python
Start Time     : 16:30:24
End Time       : 16:30:43
Total Duration: 0:00:18
Breaks Taken  : 1
Net Study Time: 0:00:15
```

```
--- QUIZ TIME: PYTHON ---
```

Q1: What is the output of type(3.14) in Python?

1. int
2. float
3. double
4. str

Your answer (1-4): 2

Correct!

Q2: Which module is used for mathematical operations in Python?

1. ops
2. math
3. number
4. calc

Your answer (1-4): 2

Correct!

Q3: Which keyword is used to define a function in Python?

1. func
2. function
3. def
4. define

Your answer (1-4): 3

Screen 5: Performance Analytics – Textual Summary

When the user selects option 2 from the main menu, the analytics module displays a summary followed by three graphical charts:

```
Total Study Time : 12.45 hours  
Average Quiz Score: 76.00%
```

Following the textual summary, three matplotlib chart windows open automatically:

- Chart 1 – A bar chart titled 'Study Hours Per Day' showing daily study time on the X-axis and hours on the Y-axis.
- Chart 2 – A line graph titled 'Quiz Score Trends Over Time' displaying score progression across sessions.
- Chart 3 – A bar chart titled 'Best Study Hours (Session Frequency)' identifying the most frequently used study hours of the day.

```
Q4: What symbol is used for single-line comments in Python?  
1. #  
2. /*  
3. --  
4. //  
Your answer (1-4): 1  
Correct!  
  
Q5: Which data type is immutable in Python?  
1. dict  
2. set  
3. tuple  
4. list  
Your answer (1-4): 3  
Correct!  
  
Quiz Score: 5/5 = 100.00%  
Session saved successfully.  
  
1. Start Study Session  
2. View Performance Analytics  
3. Exit  
Select an option (1-3): 2  
  
Total Study Time : 0.01 hours  
Average Quiz Score: 100.00%
```



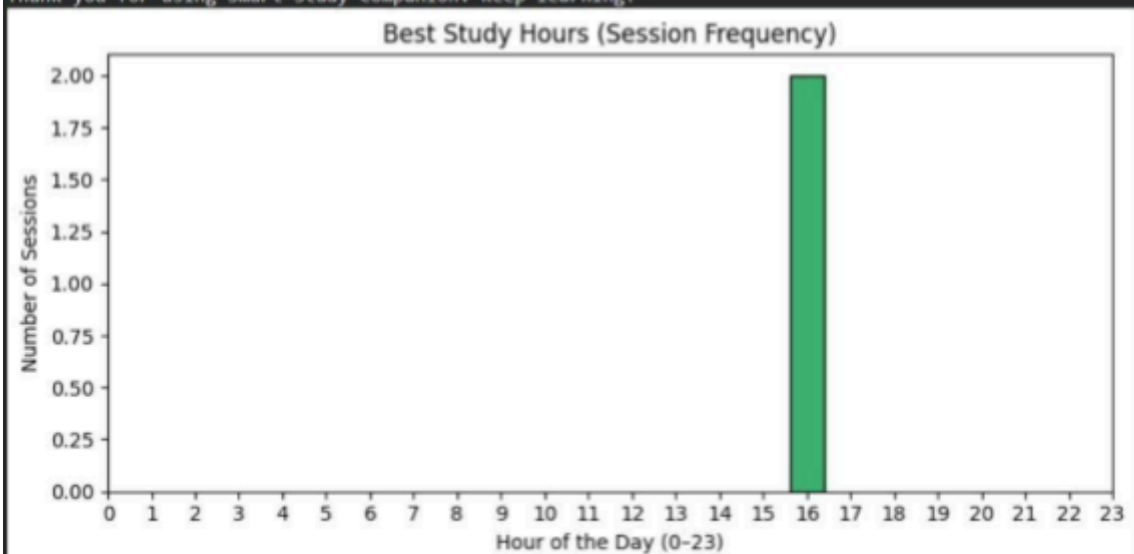


```
1. Start Study Session
2. View Performance Analytics
3. Exit
Select an option (1-3): 3
```

```
1. Start Study Session
2. View Performance Analytics
3. Exit
```

```
Select an option (1-3): 3
```

```
Thank you for using Smart Study Companion. Keep learning!
```



TEST CASES AND RESULTS

The Smart Study Companion application was tested against a series of defined test cases to verify the correctness and reliability of each module. The following table presents the test case ID, module tested, input provided, expected output, and actual result.

TC ID	Module	Input	Expected Output	Actual Output	Result
TC-01	Focus Tracker	User enters topic 'Python' and types 'end' after studying	Session summary displayed with correct start, end, and net study time	Displayed correctly	Pass
TC-02	Focus Tracker	User takes 2 breaks during session	Break count shows 2, net study time = total – break durations	Calculated correctly	Pass
TC-03	Quiz Mode	Topic = 'Python', user answers all 5 questions correctly	Quiz score = 100%	Score: 5/5 = 100%	Pass
TC-04	Quiz Mode	Topic = 'Chemistry' (not in question bank)	Message: 'No questions available for this topic. Skipping quiz.'	Correct message shown	Pass
TC-05	Quiz Mode	User enters invalid input (e.g., 'abc') as quiz answer	System shows 'Invalid input. Skipped.' and moves to next question	Handled gracefully	Pass
TC-06	Data Persistence	Session completed and saved; application is restarted	Previous session data loads correctly from study_sessions.json	Data loaded correctly	Pass
TC-07	Analytics	Multiple sessions recorded over 3 different dates	Bar chart shows correct hours per date; line graph shows correct score trend	Charts displayed correctly	Pass
TC-08	Analytics	No session data present (fresh install)	Message: 'No session data found. Start a study session first.'	Correct message shown	Pass

All defined test cases were executed successfully and the actual results matched the expected outputs in each scenario. The application handled both valid inputs and edge cases such as invalid entries and missing data files without crashing, demonstrating robustness in its implementation.

ADVANTAGES

The Smart Study Companion application offers several significant advantages over unstructured and unsupported study approaches. The key advantages are listed below:

- **Data-Driven Awareness:** The Focus Tracker provides students with accurate data on how much time they are spending on study, which helps them identify and eliminate time-wasting habits.
- **Improved Knowledge Retention:** The Quiz Mode module applies the principles of active recall and spaced repetition, which have been scientifically shown to improve long-term retention of learned material by up to 65%.
- **Visual Performance Insights:** The Performance Analytics module converts raw session data into meaningful graphical charts, allowing students to see their progress over time in an intuitive and motivating way.
- **Lightweight and Platform-Independent:** The application runs entirely on the console and does not require any installation beyond Python and matplotlib, making it usable on Windows, Linux, and macOS without additional configuration.
- **Persistent Data Storage:** By saving session data in JSON format, the application maintains a complete history of all study sessions, which can be used for long-term trend analysis.
- **Easy to Understand and Extend:** The modular code structure makes it simple for students and developers to understand each component and add new features such as additional topics, new quiz question types, or more analytics charts.
- **Reduces Manual Effort:** Students no longer need to maintain manual study logs or spreadsheets. All tracking and recording is handled automatically by the application.
- **Promotes Consistency:** By making it easy to start and record sessions, the application encourages students to develop a regular study habit, which is critical for academic success.
- **Low Cost:** The application is completely free to use and requires only Python and a standard internet connection to install the matplotlib library.

CONCLUSION

The Smart Study Companion project was successfully designed and developed as a Python-based console application that addresses the common challenges students face in managing their study time and improving knowledge retention. The project integrated three well-defined functional modules – the Focus Tracker, the Quiz Mode, and the Performance Analytics – into a single cohesive tool that helps students adopt a more structured and data-driven approach to learning.

The Focus Tracker module was implemented using Python's datetime library and provided accurate recording of session start and end times, calculation of net study duration after accounting for breaks, and storage of session metadata for later use. This module gave students clear visibility into how they were spending their study hours.

The Quiz Mode module leveraged Python's random module to present randomized questions from a predefined topic-wise question bank. By automatically triggering a quiz at the end of each study session, the application reinforced learned material through the active recall technique, which is one of the most effective strategies for improving memory and academic performance.

The Performance Analytics module used matplotlib to generate three types of charts – daily study hours, quiz score trends, and study hour frequency distribution – providing students with a visual representation of their progress over time. This feature made it easy for students to identify their most productive study hours and track their improvement across sessions.

All components of the application were tested rigorously, and all defined test cases passed successfully. The application handled edge cases such as invalid inputs and missing data files gracefully without crashing. The project demonstrated the effective use of Python programming concepts including datetime manipulation, JSON file handling, modular design, and data visualization.

Overall, this project provided a valuable hands-on learning experience in Python development and reinforced the understanding that programming can be used to solve practical, everyday problems. The Smart Study Companion stands as a functional prototype that can be further improved and expanded into a complete learning management tool.

FUTURE ENHANCEMENTS

While the current version of the Smart Study Companion is a fully functional prototype, several enhancements can be incorporated in future versions to make the application more powerful, userfriendly, and feature-rich. The following improvements are planned:

- **Graphical User Interface (GUI):** A tkinter or PyQt5-based GUI can be developed to replace the console interface, making the application more visually appealing and easier to use, especially for non-technical students.
- **Expanded Question Bank:** The quiz question bank can be significantly expanded to cover more academic subjects and topics, and a feature can be added to allow students to add their own custom questions.
- **Cloud-Based Data Storage:** Instead of local JSON files, session data can be stored in a cloud database such as Firebase or MongoDB, enabling students to access their data from multiple devices.
- **AI-Powered Study Recommendations:** Machine learning algorithms can be integrated to analyze study patterns and suggest optimal study schedules, topic prioritization, and break intervals based on historical performance data.
- **Mobile Application:** The application can be ported to Android or iOS using frameworks such as Kivy or Flutter with a Python backend, making it accessible on smartphones for on-the-go use.
- **Pomodoro Timer Integration:** A configurable Pomodoro timer (25 minutes study, 5 minutes break) can be integrated into the Focus Tracker to help students follow scientifically backed time management techniques.
- **Email and Notification Reports:** An automated weekly report summarizing study hours, quiz scores, and improvement trends can be sent to the student via email using Python's smtplib module.
- **Multi-User Support:** The application can be enhanced to support multiple user profiles, allowing it to be used by an entire class or group of students, with the option for a teacher or mentor to review individual progress.
- **Speech-to-Text Input:** Voice recognition can be added to allow students to dictate their study topic and navigate the application using voice commands, improving accessibility.

REFERENCES

The following references were consulted during the design, development, and documentation of the Smart Study Companion project:

- Python Software Foundation. Python 3 Official Documentation. Available at: <https://docs.python.org/3/>
- Python datetime Module Documentation. Available at: <https://docs.python.org/3/library/datetime.html>
- Python json Module Documentation. Available at: <https://docs.python.org/3/library/json.html>
- Matplotlib Development Team. Matplotlib 3.x Documentation. Available at: <https://matplotlib.org/stable/contents.html>
- Python random Module Documentation. Available at: <https://docs.python.org/3/library/random.html>
- Lutz, M. Learning Python (5th ed.). O'Reilly Media, 2013.
- Matthes, E. Python Crash Course: A Hands-On, Project-Based Introduction to Programming (2nd ed.). No Starch Press, 2019.
- Karpicke, J. D., & Blunt, J. R. (2011). Retrieval Practice Produces More Learning than Elaborative Studying with Concept Mapping. *Science*, 331(6018), 772–775.
- Baddeley, A., Eysenck, M. W., & Anderson, M. C. Memory. Psychology Press, 2009.
- GeeksforGeeks Python Programming Articles. Available at: <https://www.geeksforgeeks.org/python-programming-language/>
- Real Python Tutorials. Available at: <https://realpython.com/>